Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251172**
Nama Lengkap	**Ang, Joshua Alexander Hartono**
Minggu ke / Materi	**09 / Pengolahan String dan Regular Expression**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pengantar String

String adalah kumpulan karakter yang digabung menjadi satu kesatuan dan digunakan dalam program untuk menyimpan teks berupa kata atau kalimat. Data string biasanya disimpan dalam bentuk kode ASCII sehingga setiap karakter memiliki bentuk angka tertentu di dalam komputer. Tidak semua bahasa pemrograman menyediakan tipe data string secara langsung, seperti bahasa C yang mengelola string sebagai kumpulan karakter. Hal ini menunjukkan bahwa string bukan tipe data dasar, karena terdiri lebih nilai yang digabung menjadi satu. Sehingga dalam beberapa bahasa pemrograman, string dianggap sebagai array atau list dari karakter, sehingga dapat diakses dan diolah perkarakter di dalamnya.

Pengaksesan String dan Manipulasi String

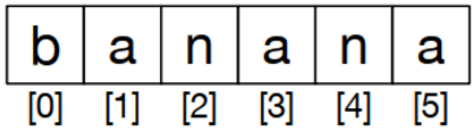
```
namasaya = "Antonius Rachmat C"
temansaya1 = "Yuan Lukito"
temansaya2 = 'Laurentius Kuncoro'
temansaya3 = "Matahari" + 'Bakti'

print(temansaya3)
print(namasaya[0]) #'A'
print(namasaya[9]) #'R'
print(temansaya1[1]) #'u'

huruf = temansaya2[0]
print(huruf) #'L'
```

Gambar 8.1: Contoh penggunaan string. *Sumber:* https://shorturl.at/DaH5D

String dapat dibuat dengan cara sederhana, yaitu dengan mendeklarasikan variabel lalu langsung mengisinya dengan sebuah teks. Dalam Python, penulisan string dapat menggunakan tanda kutip dua atau kutip satu, keduanya memiliki fungsi yang sama. Selain itu, dalam penggunaannya string dapat digabungkan menggunakan operator penjumlahan dan juga dapat diulang beberapa kali menggunakan operator bintang.

String dapat diakses sebagai satu bagian utuh dengan memanggil nama variabelnya, atau diakses per karakter dengan menggunakan indeks. Indeks pada string dimulai dari angaka 0, sehingga karakter pertama berada pada posisi ke-0. Dengan menggunakan indeks, kita bisa mengambil

huruf tertentu dari suatu string. Hal yang perlu diingat adalah indeks harus berupa bilangan bulat dan tidak bisa menggunakan pecahan.



Gambar 8.2: Bentuk string di dalam memory. *Sumber:* https://shorturl.at/DaH5D

Operator dan Metode String

Operator in adalah operator yang berfungsi untuk mengecek apakah suatu substring terdapat dalam string lain. Hasil dari pengecekan ini adalah nilai berupa True jika nilai ditemukan atau False jika nilai tidak ditemukan. Selain itu, string juga dapat dibandingkan menggunakan operator perbandingan seperti == atau >.

```
kalimat = "saya mau makan"
data = "saya"
print(data in kalimat)  #True
print("mau" in kalimat) #True
```

Gambar 8.3: Contoh penggunaan operator in. *Sumber:* https://shorturl.at/DaH5D

Fungsi len() digunakan untuk mengetahui jumlah karakter dalam sebuah string. Karena indeks string dimulai dari 0, maka untuk mengambil karakter terakhir harus menggunakan indeks len(string)-1.

```
kalimat = "universitas kristen duta wacana yogyakarta"
print(len(kalimat)) #output 42

terakhir = kalimat[len(kalimat)-1]
print(terakhir) #output 'a'

#bisa juga menggunakan indeks -1
terakhir_versi2 = kalimat[-1]
print(terakhir_versi2)  #output 'a'
#atau menggunakan indeks -2 untuk huruf terakhir kedua
terakhir2 = kalimat[-2]
print(terakhir2)    #output 't'
```

Gambar 8.4: Contoh penggunaan fungsi len. *Sumber:* https://shorturl.at/DaH5D

Traversing string adalah proses menampilkan indeks perkarakter. Hal ini dapat dilakukan menggunakan perulangan while dengan bantuan indeks atau menggunakan for yang langsung mengakses setiap karakter secara otomatis.

```
kalimat = "indonesia jaya"
i = 0
while i < len(kalimat):
    print(kalimat[i],end='')
    i += 1
```

```
kalimat = "indonesia jaya"
for kal in kalimat:
    print(kal,end='')
```

Gambar 8.5 dan 8.6: Contoh penggunaan traversing string. *Sumber:* https://shorturl.at/DaH5D

String slice digunakan untuk mengambil sebagian string dengan menentukan indeks awal dan akhir. Penulisan slice menggunakan format [awal:akhir], di mana bagian akhir tidak akan ikut diambil.

```
kalimat = "cerita rakyat"
awal = 0
akhir = 6
print(kalimat[awal:akhir])  #cerita
print(kalimat[7:len(kalimat)])  #rakyat
print(kalimat[:5])  #cerit
print(kalimat[5:])  #a rakyat
print(kalimat[:])   #cerita rakyat
```

Gambar 8.7: Contoh penggunaan string slice. *Sumber:* https://shorturl.at/DaH5D

String merupakan data yang bersifat immutable! Immutable adalah bahwa data tersebut tidak bisa diubah saat program berjalan, hanya bisa diinisialisasi saja. Contoh:

```
kalimat = "satu"
kalimat[0] = "batu" #TypeError: 'str' object does not support item assignment
```

Agar bisa diubah, maka harus disimpan dalam variabel yang berbeda.

```
kalimat = "satu"
kalimat_baru = kalimat[0] + "alah"  #salah
```

Gambar 8.8: Contoh penggunaan string data yang bersifat immutable. *Sumber:* https://shorturl.at/DaH5D

Operator + pada string berfungsi untuk menggabungkan dua buah string dan operator * berfungsi untuk menampilkan string sesuai jumlah yang diinginkan.

```
kata1 = "saya"
kata2 = "makan"
kata3 = kata1 + " " + kata2
print(kata3)    #hasil adalah penggabungan: saya makan
kata4 = "ulang"
print(kata4 * 4)    #hasil adalah ulangulangulangulang
kata4 = "ulang "
print(kata4 * 2)    #hasil adalah ulang ulang
```

Gambar 8.9: Contoh penggunaan operator * dan + pada string. *Sumber:* https://shorturl.at/DaH5D

Nama Method	Kegunaan	Penggunaan
capitalize()	untuk mengubah string menjadi huruf besar	string.capitalize()
count()	menghitung jumlah substring yang muncul dari sebuah string	string.count()
endswith()	mengetahui apakah suatu string diakhiri dengan string yang dinputkan	string.endswith()
startswith()	mengetahui apakah suatu string diawali dengan string yang dinputkan	string.startswith()
find()	mengembalikan indeks pertama string jika ditemukan string yang dicari	string.find()
islower() dan isupper()	mengembalikan True jika string adalah huruf kecil / huruf besar	string.islower() dan string.isupper()
isdigit()	mengembalikan True jika string adalah digit (angka)	string.isdigit()
strip()	menghapus semua whitespace yang ada di depan dan di akhir string	string.strip()
split()	memecah string menjadi token-token berdasarkan pemisah, misalnya berdasarkan spasi	string.split()

Gambar 8.10: Method yang sering digunakan untuk mengembalikan string. *Sumber:* https://shorturl.at/DaH5D

Parsing String

Parsing string adalah proses untuk menelusuri dan mengolah bagian-bagian dari sebuah string untuk mendapatkan informasi tertentu sesuai kebutuhan. Teknik ini biasanya digunakan untuk mengambil, memisahkan, atau mengubah isi string menjadi format yang diinginkan.

```
kalimat = "Saudara-saudara, pada tanggal 17-08-1945 Indonesia merdeka"

hasil = kalimat.split(" ")

for kal in hasil:
    if kal[0].isdigit():
        hasil2 = kal.split("-")
        print(hasil2[1]+"/"+hasil2[0]+"/"+hasil2[2])
```

Hasil dari keluaran program adalah **08/17/1945**

Gambar 8.9: Contoh penggunaan parsing string. *Sumber:* https://shorturl.at/DaH5D

Pengantar Regex

Regex atau regular expression adalah teknik yang digunakan untuk mempermudah pengolahan string dengan cara me cocokkan pola tertentu. Dibandingkan dengan metode string biasa seperti split() atau find(), regex memberikan cara yang lebih cepat dan fleksibel dalam mencari, mengganti atau menghapus bagian dari string.

```
import re
handle=open('mbox-short.txt')
count = 0
for line in handle:
line=line.rstrip()
if re.search('From:', line):
count += 1
print(line)
print("Count: ",count)
```

Gambar 8.10: Contoh penggunaan regex pada file mbox-short.txt untuk menghasilkan semua string yang terdapat pola “Form: ”. *Sumber:* https://shorturl.at/DaH5D

Meta Character, Escaped Character, Set of Character, dan Fungsi Regex pada Libary Python

Karakter	Kegunaan	Contoh	Arti Contoh
[]	Kumpulan karakter	"[a-zA-Z]"	1 karakter antara a-z kecil atau A-Z besar
\{}	Karakter dengan arti khusus dan escaped character	\{}d	Angka / digit
.	Karakter apapun kecuali newline	say.n.	Tidak bisa diganti dengan karakter apapun, misal "sayang" akan valid
^	Diawali dengan	^From	Diawali dengan From
$	Dakhiri dengan	this$	Diakhiri dengan kata this
*	0 s/d tak terhingga karakter	\{}d*	ada digit minimal 0 maksimal tak terhingga
?	ada atau tidak (opsional)	\{}d?	Boleh ada atau tidak ada digit sebanyak
+	1 s/d tak terhingga karakter	\{}d+	Minimal 1 s/d tak terhingga karakter
{}	Tepat sebanyak yang ada para {}	\{}d{2}	Ada tepat 2 digit
()	Pengelompokan karakter / pola	(saya|kamu)	saya atau kamu sebagai satu kesatuan
|	atau	\{}d|\{}s	1 digit atau 1 spasi

Special Characters	Kegunaan	Contoh
\b	Digunakan untuk mengetahui apakah suatu pola berada di awal kata atau akhir kata	"R\bin" "Rain\b"
\d	Digunakan untuk mengetahui apakah karakter adalah sebuah digit (0 s/d 9)	\d
\D	Digunakan untuk mengetahui apakah karakter yang bukan digit	\D
\s	Digunakan untuk mengetahui apakah karakter adalah whitespace (spasi, tab, enter)	\s
\S	Digunakan untuk mengetahui apakah karakter adalah BUKAN whitespace (spasi, tab, enter)	\S
\w	Digunakan untuk mengetahui apakah karakter adalah word (a-z, A-Z, 0-9, dan _)	\w
\W	Digunakan untuk mengetahui apakah karakter adalah BUKAN word (a-z, A-Z, 0-9, dan _)	\W
\A	Digunakan untuk mengetahui apakah karakter adalah berada di bagian depan dari kalimat	"\AThe"
\Z	Digunakan untuk mengetahui apakah karakter adalah berada di bagian akhir dari kalimat	"End\Z"

[abc]	Mencari pola 1 huruf a, atau b, atau c
[a-c]	Mencari pola 1 huruf a s/d c
[^bmx]	Mencari pola 1 huruf yang bukan b,m, atau x
[012]	Mencari pola 1 huruf 0, atau 1, atau 2
[0-3]	Mencari pola 1 huruf 0 s/d 3
[0-2][1-3]	Mencari pola 2 huruf: 01, 02, 03, 11, 12, 13, 21, 22, 23
[a-zA-Z]	Mencari pola 1 huruf a-Z

Nama Fungsi	Kegunaan
findall	mengembalikan semua string yang sesuai pola (matches)
search	mengembalikan string yang sesuai pola (match)
split	memecah string sesuai pola
sub	mengganti string sesuai dengan pola yang cocok

Gambar 8.11, 8.12, 8.13, dan 8.14: Tabel special character, escaped character, himpunan karakter, dan fungsi regex pada Python. *Sumber: https://shorturl.at/DaH5D*

BAGIAN 2: LATIHAN MANDIRI (60%)

Link github : https://github.com/angjoshua-sudo/Prak-Alpro-2026.git

SOAL 1

Source code:

```
1   kata_1 = input("Masukkan kata 1: ")
2   kata_2 = input("Masukkan kata 2: ")
3
4   kata_1 = ''.join([i for i in kata_1 if i.isalpha()])
5   kata_1 = kata_1.lower()
6   kata_2 = ''.join([i for i in kata_2 if i.isalpha()])
7   kata_2 = kata_2.lower()
8
9   if sorted(kata_1) == sorted(kata_2):
10      print("kata 1 dan kata 2 adalah anagram.")
11  else:
12      print("kata 1 dan kata 2 bukan anagram.")
13
```

Penjelasan:

Pada baris pertama dan baris kedua, program meminta pengguna untuk memasukkan dua kata yang ingin dibandingkan anagram atau tidak. Kedua input ini kemudian akan disimpan ke dalam variabel **kata_1** dan **kata_2**. Pada baris ke-4, **kata_1** diproses dengan mengambil hurufnya menggunakan **isalpha()**, selain hurufnya akan diabaikan. Setelahnya akan digabung kembali menggunakan **join()**. Kemudian pada baris kelima semua huruf akan diubah menjadi huruf kecil menggunakan **lower()** untuk mengantisipasi terdapatnya huruf kapital. Sehingga perbandingannya tidak sensitif terhadap huruf kapital atau nonkapital. Proses pada baris keenam dan ketujuh sama seperti pada baris keempat dan kelima tetapi dilakukan pada **kata_2**.

Setelahnya, program akan membandingkan kedua kata pada baris kesembilan. Pada baris ini terdapat **sorted()** yang berfungsi untuk mengurutkan huruf yang ada. Jika hasil urutan dari kata_1 dan kata_2 sama maka kedua kata memiliki jumlah huruf yang sama dan hanya berbeda urutan. Sehingga, baris kesepuluh akan dicetak dan menampilkan bahwa “kata 1 dan kata 2 adalah anagram”. Namun, jika tidak sama maka baris kedua belas akan dijalankan dan mencetak “kata 1 dan kata 2 bukan anagram”.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Acer/OneDrive/Dokumen/belajar-gi
thub/minggu 9/soal1.py"
Masukkan kata 1: mata
Masukkan kata 2: aatm
kata 1 dan kata 2 adalah anagram.
```

SOAL 2

Source code:

```python
kalimat = input("masukkab kalimat: ")
cari = input("masukkan jumlah kata yang ingin dicari: ")
hasil = 0

kalimat = kalimat.lower()
kalimat = kalimat.split(" ")
cari = cari.lower()

for kata in kalimat:
    kata = ''.join([i for i in kata if i.isalpha()])
    if kata == cari:
        hasil += 1

print(F'kata {cari} ada {hasil} buah')
```

Penjelasan:

Pada baris pertama dan kedua program meminta input dari pengguna berupa sebuah kalimat dan kata yang ingin dicari jumlah munculnya. Input pertama disimpan dalam variabel **kalimat** sedangkan input kedua dimasukkan ke variabel **cari**. Lalu pada baris ketiga terdapat variabel **hasil** yang bernilai 0 yang berfungsi untuk menampung jumlah kemunculan kata yang dicari nanti.

Pada baris kelima dan baris keenam, **kalimat** akan diubah menjadi huruf kecil semua dengan **lower()**, lalu dipisahkan menggunakan **split(“ ”)** menjadi list kata-kata. Hal ini bertujuan agar perbandingan tidak terpengaruh oleh huruf kapital dan dapat dicek perkata. Pada baris kesembilan sampai dengan baris kedua belas terdapat perulangan kata untuk melihat setiap kata dalam list **kalimat**. Di dalam loop, setiap kata dibersihkan menggunakan **isalpha()** agar hanya menyisakan huruf saja, jadi tanda baca akan dihapus. Setelah bersih, kata akan dibandingkan pada baris kesebelas. Jika kata yang dibandingkan sama maka variabel **hasil** akan bertambah 1. Terakhir pada baris keempat belas, program akan mencetak hasil progam menggunakan f-string.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Acer/OneDrive/Dokumen/belajar-gi
thub/minggu 9/soal2.py"
masukkab kalimat: Saya mau makan. Makan itu wajib. Mau siang atau malam saya wajib makan
masukkan jumlah kata yang ingin dicari: makan
kata makan ada 3 buah
```

SOAL 3

Source code:

```python
kalimat = input("masukkan kalimat: ")
simpan = []



kalimat = kalimat.split(' ')

for kata in kalimat:
    if kata != '':
        simpan.append(kata)

simpan = ' '.join(simpan)
print(f'"{simpan}"')
```

Penjelasan:

Pada baris pertama, program minta input dari para user berupa kalimat dan disimpan ke variabel **kalimat**. Kemudian di baris kedua terdapat variabel **simpan** yang berisi list kosong yang nantinya akan dipakai untuk menampung kata-kata yang sudah di proses. Masuk ke baris keenam, **kalimat.split(‘ ‘)** berfungsi untuk memecah kalimat jadi list berdasarkan spasi, jadi tiap kata jadi elemen list. Di baris kedelapan sampai kesepuluh, dilakukan looping untuk tiap kata, dan dicek kalau bukan list kosong **(‘’)**, maka akan dimasukkan ke list **simpan**. Ini penting karena hasil **split** bisa menghasilkan elemen kosong kalau ada spasi ganda. Pada baris dua belas, list **simpan** digabung menjadi satu kalimat menggunakan **‘ ‘.join(simpan)** dengan satu spasi sebagai pemisah. Hasilnya disimpan ke dalam simpan dalam bentuk list. Terakhir, baris ketiga belas akan mencetak hasil dari variabel simpan menggunakan f-string.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Acer/OneDrive/Dokumen/belajar-gi
thub/minggu 9/soal3.py"
masukkan kalimat: "saya     tidak suka   memancing       ikan "
"saya tidak suka memancing ikan"
```

SOAL 4

Source code:

```
1   kalimat = input("Masukkan kalimat: ")
2
3   kalimat = kalimat.split(' ')
4
5   pendek=kalimat[0]
6   panjang=kalimat[0]
7
8   for kata in kalimat:
9       kata = ''.join([i for i in kata if i.isalpha()])
10      if len(kata) < len(pendek):
11          pendek = kata
12      if len(kata) > len(panjang):
13          panjang = kata
14
15  print(f'terpendek: {pendek}, terpanjang: {panjang}')
```

Penjelasan:

Di baris pertama, program minta input kalimat lalu disimpan di variabel kalimat. Baris ketiga, **kalimat** itu dipecah menjadi list kata-kata menggunakan acuan spasi **pakai split(‘ ‘)**. Di baris kelima dan keenam, variabel **pendek** dan **panjang** diinisialisasi dengan kata pertama sebagai nilai pembanding. Ini asumsi awal supaya nanti bisa dibandingkan dengan kata lain.

Masuk ke perulangan di baris kedelapan, program akan mengecek setiap kata dalam list **kalimat**. Di baris kesembilan, tiap kata dibersihkan dulu dengan **“”.join([i for i in kata if i.isalpha()])**, artinya hanya huruf saja yang diambil, jadi tanda baca atau simbol dibuang. Setelah itu di baris ke sepuluh dan sebelas dicek apakah panjang kata sekarang lebih kecil dari **pendek**, kalau iya maka **pendek** diperbaharui. Lalu di baris dua belas dan tiga belas, dicek juga apakah panjang kata lebih besar dari **panjang**, kalau iya maka **panjang** diperbaharui. Terakhir, kode dicetak ke layar.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Acer/OneDrive/Dokumen/belajar-gi
thub/minggu 9/soal4.py"
Masukkan kalimat: red snakes and a black frog in the pool
terpendek: a, terpanjang: snakes
```

SOAL 5

Source code:

```python
1  from datetime import datetime
2
3  kalimat = input("Masukkan kalimat: ")
4  temp = []
5
6  kalimat = kalimat.split(' ')
7  for i in kalimat:
8      simpan = i.strip(",()!?./;:'\"")
9      jadi = simpan.split("-")
10
11     if len(jadi) == 3 and all(j.isdigit() for j in jadi):
12         temp.append(simpan)
13
14 for j in temp:
15     tanggal = datetime.strptime(j, "%Y-%m-%d")
16     sekarang = datetime.now()
17
18     selisih = (sekarang - tanggal).days
19
20     ubah_tanggal = tanggal.strftime("%d-%m-%Y")
21     print(f"{ubah_tanggal} selisih {selisih} hari")
```

Penjelasan:

Di baris pertama, program mengimpor **datetime** dari library bawaan Python yang nanti dipakai untuk mengelola tanggal. Pada baris ketiga, program meminta input dari user berupa kalimat lalu disimpan ke variabel **kalimat**, sementara di baris keempat dibuat list kosong temp untuk menampung tanggal yang valid. Di baris keenam, kalimat yang diinput dipecah menjadi list kata-kata berdasarkan spasi menggunakan **split(‘ ‘)**. Kemudian pada baris ke tujuh sampai kesepuluh, program akan melakukan looping tiap kata, membersihkan tanda baca di pinggir kata dengan **strip**, lalu memisahkan kata tersebut berdasarkan tanda **–** untuk dicek apakah bentuknya seperti tanggal.

Di baris kesebelas, program akan mengecek apakah hasil split tadi berjumlah tiga bagian dan semuanya angka, yang berarti formatnya mirip tanggal (YYYY-MM-DD). Kalau memenuhi syarat, baris dua belas akan memasukkan kata tersebut ke **temp**. Lanjut ke baris keempat belas, program melakukan looping untuk setiap tanggal yang sudah lolos seleksi. Di baris kelima belas, string tanggal diubah menjadi objek datetime pakai **strptime**, lalu di baris keenam belas diambil waktu

sekarang dengan **datetime.now()**. Baris kedelapan belas menghitung selisih hari antara tanggal sekarang dan tanggal yang ditemukan. Kemudian, baris kedua puluh mengubah tanggal menjadi DD-MM-YYYY menggunakan **strftime** agar sesuai dengan permintaan soal. Terahkir pada baris dua puluh satu, program akan mencetak tanggal beserta selisih hari dengan sekarang,

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Acer/OneDrive/Dokumen/belajar-gi
thub/minggu 9/soal5.py"
Masukkan kalimat: #Pada tanggal 1945-08-17 Indonesia merdeka. Indonesia memiliki beberapa pahlawan nasional, seperti Pangeran Diponegoro (TL: 1785-11-11), Patt
imura (TL: 1783-06-08) dan Ki Hajar Dewantara (1889-05-02).
17-08-1945 selisih 29463 hari
11-11-1785 selisih 87815 hari
08-06-1783 selisih 88702 hari
02-05-1889 selisih 50023 hari
```

SOAL 6

Source code:

```python
1  import random
2  import string
3  kalimat = input("Masukkan kalimat: ")
4  mail =[]
5  nama = []
6
7  kata = kalimat.split()
8  for i in kata:
9      bersih = i.strip(".,!?;:'\"()")
10
11     if "@" in bersih:
12         mail.append(bersih)
13
14 for j in mail:
15     depan = j.split("@")[0]
16     nama.append(depan)
17
18     karakter = string.ascii_letters + string.digits
19     password = ''
20
21     for l in range(8):
22         password += random.choice(karakter)
23
24     print(f'{j} username: {depan} , password: {password}')
```

Penjelasan:

Di baris pertama dan kedua, program mengimpor dua library yaitu **random** untuk menghasilkan nilai acak dan **string** untuk mengambil kumpulan karakter seperti huruf dan angka. Pada baris ketiga, program meminta input berupa **kalimat** dari user lalu disimpan di variabel kalimat. Di

baris keempat dan kelima dibuat dua list kosong, yaitu **mail** untuk menyimpan nama email dan **nama** untuk menyimpan bagian username dari email. Di baris ketujuh, kalimat dipecah menjadi kata-kata menggunakan **split()**.

Di baris kedelapan sampai kesepuluh, program melakukan perulangan untuk setiap kata, lalu membersihkan tanda baca di awal atau di akhir kata menggunakan **strip**. Setelah itu di baris kesebelas, dicek apakah dalam kata tersebut ada simbol '@' yang menandakan email. Jika ada maka di baris dua belas kata tersebut akan dimasukkan ke dalam list mail. Jika ada, maka di baris kedua belas kata tersebut dimasukkan ke dalam list **mail**. Jadi bagian ini berfungsi untuk menyaring kata yang kemungkinan adalah email.

Di baris keempat belas, program melakukan looping lagi untuk setiap email yang sudah ditemukan. Baris kelima belas, email tersebut dipisahkan berdasarkan tanda "@" dan diambil bagian depannya saja sebagai username. Lalu di baris keenam belas, username itu disimpan ke dalam list **nama**. Jadi di sini program mengekstrak nama pengguna dari email.

Di baris kedelapan belas, dibuat variabel **karakter** yang berisi gabungan huruf dan angka dari **string.ascii_letters** dan **string.digits**. Baris kesembilan belas, variabel **password** diinisaialisasi sebagai string kosong. Kemudian di baris kedua puluh sampai dua puluh dua, program membuat password acak sepanjang 8 karakter dengan mengambil karakter secara acak menggunakan **random.choice**. Terakhir di baris dua puluh empat, program mencetak email, username, dan password yang sudah dihasilkan.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Acer/OneDrive/Dokumen/belajar-gi
thub/minggu 9/soal6.py"
Masukkan kalimat: #Berikut adalah daftar email dan nama pengguna dari mailing list: anton@mail.com dimiliki oleh antonius budi@gmail.co.id dimiliki oleh budi a
nwari slamet@getnada.com dimiliki oleh slamet slumut matahari@tokopedia.com dimiliki oleh toko matahari
anton@mail.com username: anton , password: al7OYeWa
budi@gmail.co.id username: budi , password: naTLUUtx
slamet@getnada.com username: slamet , password: 3cjdVfzS
matahari@tokopedia.com username: matahari , password: o3snJ377
```